ข้อตกลง API ระบบจองห้องพัก

Hotel Booking REST API รุ่น 1.0 วันที่ 28 ตุลาคม 2026

ข้อเสนอสำหรับทีม 3 คนใช้เป็น API contract ใน Sprint 1 เพื่อให้พัฒนา Users, Rooms และ Bookings พร้อมกันได้ ชื่อ Endpoint ฟิลด์ และกติกาที่ออกแบบเพิ่มในเอกสารนี้เป็นข้อเสนอของทีม จนกว่าสมาชิกจะตกลงใช้ร่วมกัน

สิ่งที่โจทย์กำหนด

Project Description กำหนดระบบบัญชีและสิทธิ์ User/Admin การจัดการห้อง ค้นหาห้องว่าง การจองที่ไม่ทับซ้อน สถานะ Pending เริ่มต้น ตามด้วย Approved, Cancelled และ Paid การบันทึกเหตุการณ์สร้างและยกเลิกการจอง ตลอดจน Health check และ Swagger

โจทย์ไม่ได้แจกแจงชื่อ Endpoint, Schema, ตารางเปลี่ยนสถานะ หรือระบุว่าสถานะใดกันห้องไว้ จึงใช้ข้อเสนอในหน้าถัดไปให้ทีมเริ่มทำงานได้ หากอาจารย์ชี้แจงเพิ่ม ให้แก้กติกาและ Test ที่เกี่ยวข้องพร้อมกัน

มาตรฐานที่เสนอใช้ร่วมกัน

หัวข้อ	ข้อตกลง
Base URL	/api/v1 สำหรับ Business API; /health สำหรับตรวจระบบ; /docs สำหรับ Swagger
รูปแบบข้อมูล	JSON และ camelCase; Enum ใช้ USER, ADMIN และ PENDING, APPROVED, CANCELLED, PAID
ID และวันเวลา	ID เป็น UUID string; createdAt/updatedAt เป็น UTC ISO 8601; วันเข้าพักเป็น YYYY-MM-DD
Authentication	Authorization: Bearer <accessToken>; Guest ไม่ต้องส่ง Token
Validation	ข้อมูลผิดคืน 400; ไม่รับฟิลด์ที่ไม่ได้กำหนด; PATCH ต้องมีอย่างน้อยหนึ่งฟิลด์
Pagination	page เริ่ม 1; limit ค่าเริ่มต้น 20 สูงสุด 100; เรียง createdAt DESC และ id DESC
ความสำเร็จ	มี data และ message; รายการมี meta เพิ่ม; 204 ไม่มี Body
ความผิดพลาด	มี error.code, error.message, error.details และ requestId ตามหัวข้อ 4

แหล่งอ้างอิง: 69-1-ITCS258-Project Description จาก Pasted text.txt ที่ทีมได้รับ โดยเฉพาะ Task 1–8 และ Marking Criteria ใช้อ้างอิงเป็นหมวด FR โดยไม่แต่งหมายเลขรายข้อที่ต้นฉบับไม่ได้แจกแจง

1 บัญชีผู้ใช้และการยืนยันตัวตน

เจ้าของหลัก คนที่ 1 | อ้างอิงหมวด FR-1–FR-7 และ Task 2–3

Method และ Path	สิทธิ์	Request	ผลสำเร็จ
POST /auth/register	Guest	name, email, password	201 User
POST /auth/login	Guest	email, password	200 Token และ User
POST /auth/logout	Login	ไม่มี Body	204
GET /users/me	Login	ไม่มี Body	200 User
PATCH /users/me	Login	name และ/หรือ email	200 User

Path ในตารางนี้และตาราง Business API ทุกหน้าต้องเติม /api/v1 ข้างหน้า คำว่า Login ครอบคลุม USER และ ADMIN

Request และฟิลด์ของ User

Register: name เป็น string ตัดช่องว่างหัวท้าย ยาว 1–100 ตัวอักษร; email รูปแบบอีเมล ยาวไม่เกิน 254 และ Normalize เป็นตัวพิมพ์เล็ก; password ยาว 8–128 ตัวอักษร โดยไม่ Trim รหัสผ่าน ทั้งสามฟิลด์จำเป็น

Login ใช้ email และ password ตามกติกาเดียวกัน ส่วน PATCH /users/me รับเฉพาะ name/email ไม่รับ role หรือ password การเปลี่ยนรหัสผ่านยังไม่เพิ่มในขอบเขตนี้

```
{
  "data": {
    "id": "<uuid>",
    "name": "Demo User",
    "email": "user@example.com",
    "role": "USER",
    "createdAt": "2026-10-28T09:00:00Z",
    "updatedAt": "2026-10-28T09:00:00Z"
  },
  "message": "User registered"
}
```

Login คืน data = { accessToken, tokenType: "Bearer", expiresIn: 900, user: User } ไม่คืน passwordHash หรือข้อมูล Secret ในทุก Response

JWT และ Logout ที่เสนอ

ใช้ Access token อายุ 15 นาที มี sub = userId, role, jti, iat และ exp; ไม่มี Refresh token ในรุ่นนี้ Register สร้างได้เฉพาะ USER ส่วน ADMIN สร้างผ่าน Seed/คำสั่งที่ควบคุมไว้และไม่เก็บรหัสผ่านจริงใน Repository

Logout เพิกถอน Token ปัจจุบัน โดยเก็บ jti ใน Redis จนถึง exp และ Guard ตรวจทุก Protected request หากบริการตรวจเพิกถอนใช้งานไม่ได้ ให้คืน 503 เพื่อไม่ยอมรับ Token ที่อาจถูกเพิกถอน Logout ไม่ได้ออกจากระบบทุกอุปกรณ์

ข้อผิดพลาดสำคัญ

409 EMAIL_ALREADY_EXISTS เมื่ออีเมลซ้ำ; 401 INVALID_CREDENTIALS เมื่อ Login ไม่สำเร็จโดยใช้ข้อความเดียวกันทั้งกรณีไม่พบบัญชีและรหัสผ่านผิด; 401 UNAUTHORIZED เมื่อ Token หาย ผิด หมดอายุ หรือถูกเพิกถอน; 429 RATE_LIMITED เมื่อเกินขีดจำกัดที่ตั้งไว้

2 ห้องพักและการค้นหาห้องว่าง

เจ้าของหลัก คนที่ 2 | อ้างอิงหมวด FR-8–FR-16 และ FR-27–FR-29

Method และ Path	สิทธิ์	Request	ผลสำเร็จ
GET /rooms	ทุกคน	page, limit	200 รายการห้อง Active
GET /rooms/search	ทุกคน	checkIn, checkOut, guests; page, limit	200 ห้องว่าง
GET /rooms/{roomId}	ทุกคน	UUID ใน Path	200 Room ที่ Active
GET /admin/rooms	ADMIN	page, limit; isActive ไม่บังคับ	200 รวม Active/Inactive
GET /admin/rooms/{roomId}	ADMIN	UUID ใน Path	200 Room ทุกสถานะ
POST /rooms	ADMIN	Room input ครบ	201 Room
PATCH /rooms/{roomId}	ADMIN	Room input บางฟิลด์	200 Room
DELETE /rooms/{roomId}	ADMIN	ไม่มี Body	204

Room input และ Response

Input: name string 1–100, description string สูงสุด 2,000 (ไม่บังคับ ค่าเริ่มต้น ""), capacity integer ตั้งแต่ 1, pricePerNight decimal string เช่น "1500.00" มากกว่า 0 และไม่เกิน 9,999,999.99; isActive boolean ค่าเริ่มต้น true; images array ค่าเริ่มต้น []

แต่ละ Image มี url แบบ HTTPS, altText string ไม่เกิน 200 และ sortOrder integer ตั้งแต่ 0 การ PATCH images คือแทนที่ทั้ง Array ใช้ข้อมูลอ้างอิงรูปภาพ ไม่เพิ่มระบบอัปโหลดไฟล์ในรุ่นนี้

Room response มี id, name, description, capacity, pricePerNight, currency = "THB", isActive, images, createdAt และ updatedAt ใช้เงินเป็น Decimal ใน Database และส่งเป็น string ไม่คำนวณยอดเงินด้วย float

Search และการจัดการห้อง

Search ต้องส่ง checkIn/checkOut/guests ครบ guests เป็น integer ตั้งแต่ 1 คืนเฉพาะห้อง Active ที่ capacity ≥ guests และไม่มีการจองที่กันห้องไว้ทับซ้อน เรียงและแบ่งหน้าเหมือน GET /rooms; ไม่พบห้องคืน 200 พร้อม data = [] และ total = 0

GET /rooms ไม่รับตัวกรองวันที่ ให้ใช้ /rooms/search และประกาศ Route search ก่อน Route {roomId} ส่วน Public detail ของห้อง Inactive คืน 404

ห้องมีประวัติการจองใด ๆ ให้ DELETE คืน 409 ROOM_HAS_BOOKINGS และใช้ PATCH isActive=false แทน การปิดห้องไม่ยกเลิกการจองเดิม แต่ห้ามสร้างใหม่ การลบห้องที่ไม่มีประวัติทำได้จริง

แก้ราคามีผลกับการจองใหม่เท่านั้น หากลด capacity ต่ำกว่า guests ของการจองที่กันห้องไว้และยังไม่สิ้นสุด ให้คืน 409 ROOM_CAPACITY_CONFLICT

3 การจองและขอบเขตสิทธิ์

เจ้าของหลัก คนที่ 3 | อ้างอิงหมวด FR-17–FR-26

Method และ Path	สิทธิ์	Request	ผลสำเร็จ
POST /bookings	Login	roomId, checkIn, checkOut, guests	201 Booking
GET /bookings	Login	page, limit; status ไม่บังคับ	200 ของตัวเอง
GET /bookings/{bookingId}	เจ้าของ	UUID ใน Path	200 Booking
PATCH /bookings/{bookingId}/status	เจ้าของ	status: "CANCELLED" เท่านั้น	200 Booking
GET /admin/bookings	ADMIN	page, limit; status, userId, roomId ไม่บังคับ	200 ทั้งระบบ
GET /admin/bookings/{bookingId}	ADMIN	UUID ใน Path	200 Booking
PATCH /admin/bookings/{bookingId}/status	ADMIN	status ตามตารางหัวข้อ 7	200 Booking

Request สร้างการจอง

```
{
  "roomId": "<uuid>",
  "checkIn": "2026-11-10",
  "checkOut": "2026-11-12",
  "guests": 2
}
```

roomId เป็น UUID; guests เป็น integer ตั้งแต่ 1 และไม่เกินความจุห้อง; ห้าม Client ส่ง userId, status, price หรือ totalPrice ระบบรับ userId จาก Token และคำนวณราคาเอง วันที่ในตัวอย่างให้ใช้กับเวลาทดสอบที่ตรึงไว้เป็น 2026-10-28

Booking response

```
{
  "data": {
    "id": "<booking-uuid>",
    "userId": "<user-uuid>",
    "roomId": "<room-uuid>",
    "checkIn": "2026-11-10",
    "checkOut": "2026-11-12",
    "guests": 2,
    "status": "PENDING",
    "nights": 2,
    "pricePerNight": "1500.00",
    "totalPrice": "3000.00",
    "currency": "THB",
    "createdAt": "2026-10-28T09:00:00Z",
    "updatedAt": "2026-10-28T09:00:00Z"
  },
  "message": "Booking created"
}
```

เก็บราคา ณ เวลาจองเป็น Snapshot คืน Booking schema เดียวกันใน Create/Detail/Update และคืน Array ของ Schema นี้ใน List; ไม่เปิดให้แก้ห้องหรือวันที่ของการจองเดิมในรุ่นนี้ ให้ยกเลิกและสร้างใหม่ตามเงื่อนไข

เส้นทาง /bookings ใช้ข้อมูลของตัวเอง แม้ผู้เรียกเป็น ADMIN; ADMIN ใช้ /admin/bookings เมื่อต้องจัดการผู้อื่น การร้องขอ ID ของคนอื่นผ่าน Owner route คืน 404 RESOURCE_NOT_FOUND เพื่อไม่เปิดเผยว่ามีข้อมูลนั้น

4 รูปแบบ Response และ Error

Response สำเร็จ

รายการใช้ { data: [...], message: string, meta: { page, limit, total, totalPages } } โดย total คือจำนวนที่ผ่านตัวกรองก่อนแบ่งหน้า และ totalPages = ceil(total / limit) ถ้าไม่มีข้อมูลให้เป็น 0 ส่วน Single resource ใช้ { data: object, message: string }

Error schema กลาง

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid request data",
    "details": [
      {
        "field": "checkOut",
        "message": "Must be after checkIn"
      }
    ]
  },
  "requestId": "<request-uuid>"
}
```

details เป็น Array เสมอ ถ้าไม่มีรายละเอียดใช้ []; error.code ใช้สำหรับโปรแกรม ส่วน message เป็นภาษาอังกฤษที่อ่านรู้เรื่อง ห้ามส่ง Stack trace, SQL, Password หรือ Token ใน Response/Log; ส่ง X-Request-Id ในทุก Response เพื่อจับคู่ Log

HTTP	error.code ตัวอย่าง	ใช้เมื่อ
400	VALIDATION_ERROR	JSON/Query/UUID/Enum/วันที่หรือข้อมูลผิด
401	UNAUTHORIZED / INVALID_CREDENTIALS	ยังไม่ยืนยันตัวตน หรือ Login ไม่สำเร็จ
403	FORBIDDEN	ยืนยันตัวตนแล้วแต่ไม่มีสิทธิ์ Admin
404	RESOURCE_NOT_FOUND	ไม่พบข้อมูล หรือ Owner route ไม่ใช่เจ้าของ
409	EMAIL_ALREADY_EXISTS	อีเมลซ้ำ
409	ROOM_UNAVAILABLE	ห้อง Inactive หรือมีการจองทับซ้อน
409	INVALID_STATUS_TRANSITION	เปลี่ยนสถานะ/ยกเลิกไม่ได้ตามกติกา
409	ROOM_HAS_BOOKINGS / ROOM_CAPACITY_CONFLICT	ลบหรือลดความจุไม่ได้ตามหัวข้อ 2
429	RATE_LIMITED	เกินขีดจำกัด พร้อม Retry-After เป็นวินาที
500	INTERNAL_ERROR	ข้อผิดพลาดที่ไม่คาดคิด ใช้ข้อความทั่วไป
503	SERVICE_UNAVAILABLE	Database/บริการจำเป็นไม่พร้อม

Success codes: GET/PATCH/Login ใช้ 200, Create ใช้ 201 และ Location ไปยัง Resource detail ที่ผู้เรียกมีสิทธิ์, Logout/Delete ใช้ 204 และไม่มี Body ไม่ใช้ 200 ครอบ Error

ลำดับตรวจ Protected API: Authentication ⟶ Role ⟶ รูปแบบข้อมูล ⟶ ค้นหา/Ownership ⟶ กฎธุรกิจ กรณี Login ผิดใช้ข้อความทั่วไป และอย่าทำให้แยกได้ว่าอีเมลใดมีบัญชี

5 การแจ้งเตือนและระบบสนับสนุน

Method และ Path	สิทธิ์	Request	ผลสำเร็จ
GET /notifications	Login	page, limit	200 รายการของตัวเอง
GET /health	ทุกคน	ไม่มี	200 หรือ 503
GET /docs	ทุกคน	ไม่มี	200 Swagger UI
GET /docs-json	ทุกคน	ไม่มี	200 OpenAPI JSON

เฉพาะ /notifications ใช้ Prefix /api/v1; Endpoint ระบบที่เหลือใช้ Path ตามตารางตรง ๆ และไม่ห่อ Swagger/OpenAPI ด้วย Envelope ของ Business API

Notification schema

มี id, userId, bookingId, type, message และ createdAt โดย type เป็น BOOKING_CREATED หรือ BOOKING_CANCELLED สร้างให้เจ้าของการจอง แม้ Admin เป็นผู้ยกเลิก ไม่เพิ่ม Email, Push, Mark as read หรือการแจ้งเตือนสถานะอื่นในรุ่นนี้

```
{
  "data": [
    {
      "id": "<notification-uuid>",
      "userId": "<user-uuid>",
      "bookingId": "<booking-uuid>",
      "type": "BOOKING_CREATED",
      "message": "Your booking has been created",
      "createdAt": "2026-10-28T09:00:00Z"
    }
  ],
  "message": "Notifications retrieved",
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 1,
    "totalPages": 1
  }
}
```

บันทึก Notification ใน Transaction เดียวกับการสร้างหรือเปลี่ยนเป็น CANCELLED ถ้า Rollback ต้องไม่เหลือ Notification และการส่งสถานะเดิมซ้ำต้องไม่สร้างเหตุการณ์ซ้ำ

Health check

คืน 200 เมื่อ API, Database และ Redis พร้อมใช้งาน เพราะแผนนี้ใช้ Redis ตรวจ Token ที่เพิกถอนและควบคุมการเรียก API ถ้าบริการจำเป็นไม่พร้อมคืน 503 ไม่เปิดเผย Connection string, Host ภายใน หรือ Secret

```
{
  "data": {
    "status": "ok",
    "checks": {
      "database": "up",
      "redis": "up"
    }
  },
  "message": "Service healthy"
}
```

เมื่อไม่พร้อมใช้ Schema เดิม แต่ status = "degraded" และ Check ที่เสียเป็น "down" เพื่อให้เครื่องมือตรวจสุขภาพอ่านโครงสร้างเดิมได้ Health response เป็นข้อยกเว้นจาก Error schema กลางที่ระบุไว้ชัดเจน

Cache และ Rate limiting ที่เสนอ

Cache เฉพาะ Public room list/detail อายุ 60 วินาที และ Invalidate เมื่อแก้/เพิ่ม/ปิด/ลบห้อง ไม่ใช้ Cache ตัดสินใจตอนสร้างการจอง และยังไม่ Cache Search availability ในรุ่นแรก

ค่าเริ่มต้นสำหรับทีม: Login/Register 5 คำขอต่อนาทีต่อ IP; สร้างการจอง 10 คำขอต่อนาทีต่อ userId ค่าเหล่านี้เป็นข้อเสนอที่ตั้งผ่าน Environment และอธิบายการเลือกค่าใน README

6 กติกาวันที่และการนับห้องว่าง

ทั้งหมดในหน้านี้เป็นกติกาที่เสนอ เพราะ Project Description ระบุให้ตรวจวันที่และห้ามจองทับซ้อน แต่ไม่ได้แจกแจงขอบเขตวันที่โดยละเอียด

วันเข้าพัก

รับ YYYY-MM-DD แบบวันที่ปฏิทินจริง เก็บใน Database เป็น DATE ไม่ใช้ Timestamp สำหรับ checkIn/checkOut กำหนดวันนี้จาก Asia/Bangkok ห้ามแปลงวันที่พักผ่าน UTC จนวันเลื่อน

checkIn ต้องเป็นวันนี้หรืออนาคต; checkOut ต้องมากกว่า checkIn อย่างน้อย 1 วัน; ยอมรับจองวันนี้และไม่เพิ่มเพดานระยะเข้าพักในรุ่นนี้ การจองหนึ่งรายการใช้ห้องหนึ่งห้อง

จำนวนคืน = จำนวนวันปฏิทินระหว่าง checkIn และ checkOut; totalPrice = nights × pricePerNight ณ เวลาจอง เป็น THB และทศนิยม 2 ตำแหน่ง ไม่มีภาษี ส่วนลด หรือ Payment gateway ในขอบเขตที่เสนอ

ใช้ช่วงวันที่แบบรวมวันเข้าและไม่รวมวันออก

ช่วงเข้าพักคือ [checkIn, checkOut) เช่น 10–12 พ.ย. หมายถึงพักคืนวันที่ 10 และ 11 จึงให้คนถัดไปเช็กอินวันที่ 12 ได้

ทับซ้อนเมื่อ existing.checkIn < requested.checkOut และ existing.checkOut > requested.checkIn โดยตรวจเฉพาะรายการที่กันห้องไว้ สูตรเดียวกันต้องถูกใช้ทั้ง Search และ Booking

การจองเดิม	คำขอใหม่	ผล
10–12 พ.ย.	12–14 พ.ย.	ไม่ทับซ้อน
10–12 พ.ย.	8–10 พ.ย.	ไม่ทับซ้อน
10–12 พ.ย.	11–13 พ.ย.	ทับซ้อน
10–12 พ.ย.	10–12 พ.ย.	ทับซ้อน
10–12 พ.ย.	9–13 พ.ย.	ทับซ้อน
10–12 พ.ย.	11–11 พ.ย.	400 วันที่ไม่ถูกต้อง

สถานะที่กันห้องไว้

PENDING, APPROVED และ PAID กันห้องไว้ตามช่วงวันที่ ส่วน CANCELLED ไม่กันห้อง การยกเลิกจึงทำให้ห้องว่างได้ทันทีเมื่อ Commit

PENDING ไม่หมดอายุอัตโนมัติในรุ่นนี้ เพื่อลดงาน Background job แต่ต้องยอมรับว่าการจองที่ค้างอนุมัติจะกันห้องจนยกเลิก ถ้าทีมเพิ่มเวลาหมดอายุภายหลัง ต้องแก้ Search, Conflict check และ Test พร้อมกัน

7 สถานะการจองและการยกเลิก

โจทย์กำหนดชื่อสถานะและ Pending เป็นค่าเริ่มต้น ส่วนเส้นทางเปลี่ยนสถานะและเงื่อนไขเวลาต่อไปนี้เป็นข้อเสนอของทีม

สถานะเดิม	สถานะใหม่	ใครทำได้	เงื่อนไข
สร้างใหม่	PENDING	USER หรือ ADMIN	จองให้ตัวเอง และผ่าน Validation/Conflict check
PENDING	APPROVED	ADMIN	วันนี้ไม่เกิน checkIn และผ่านการตรวจห้องทับซ้อน
PENDING	CANCELLED	เจ้าของ หรือ ADMIN	วันนี้ก่อน checkIn
APPROVED	PAID	ADMIN	วันนี้ไม่เกิน checkIn
APPROVED	CANCELLED	เจ้าของ หรือ ADMIN	วันนี้ก่อน checkIn

สถานะเดิม	สถานะใหม่	ใครทำได้	เงื่อนไข
สถานะเดิม	สถานะเดิม	ตามสิทธิ์ Route	200 ข้อมูลเดิม ไม่สร้าง Notification ซ้ำ
CANCELLED	สถานะอื่น	ไม่อนุญาต	409 INVALID_STATUS_TRANSITION
PAID	สถานะอื่น	ไม่อนุญาต	409 INVALID_STATUS_TRANSITION

การเปลี่ยนอื่นที่ไม่อยู่ในตาราง เช่น PENDING ⟶ PAID หรือ APPROVED ⟶ PENDING ไม่อนุญาต User route รับเฉพาะ CANCELLED เท่านั้น ส่งค่าอื่นคืน 403 FORBIDDEN หลังผ่านการตรวจเจ้าของ

ข้อจำกัดเรื่องวันเข้าพักที่ต้องเห็นตรงกัน

การอนุมัติและบันทึก Paid อนุญาตเมื่อ today ≤ checkIn; การยกเลิกอนุญาตเมื่อ today < checkIn; เมื่อ today > checkIn ห้ามเปลี่ยนสถานะ โดย today ใช้วันที่ Asia/Bangkok จึงยังจอง อนุมัติ และบันทึก Paid ในวันเช็กอินได้ แต่ยกเลิกในวันนั้นไม่ได้

Paid และการปิดห้อง

PAID หมายถึง Admin บันทึกว่าได้รับชำระแล้ว ไม่ได้ยืนยันธุรกรรมกับระบบชำระเงิน และรุ่นนี้ยังไม่รองรับคืนเงินหรือยกเลิกหลัง Paid

ปิดห้องเพื่อหยุดจองใหม่ได้โดยไม่ลบรายการเดิม Admin ยังอนุมัติ/บันทึก Paid ของการจองเดิมได้ตามวันที่ ส่วนการเปิดห้องใหม่ต้องไม่ทำให้เกิดการจองทับซ้อน

ข้อมูลที่ไม่ให้แก้ผ่านสถานะ

Status endpoint รับเฉพาะ status ไม่รับ userId, roomId, วันที่ หรือราคา กติกาทั้งหมดต้องบังคับใน Service ภายใน Transaction ไม่พึ่งเฉพาะ UI หรือ Role Guard

8 ข้อต่อระหว่างโมดูลและเกณฑ์ทดสอบ

ป้องกันจองซ้ำภายใต้คำขอพร้อมกัน

คนที่ 3 ทำการสร้าง Booking ใน Database transaction โดยล็อกแถว Room ของห้องนั้นด้วย Write lock ก่อนอ่าน Active/Capacity/ราคาและตรวจการจองทับซ้อน จากนั้น Insert Booking และ Notification แล้ว Commit หากไม่ผ่านให้ Rollback และคืน 409 หรือ 400 ตามสาเหตุ

ทุกเส้นทางที่เปลี่ยนข้อมูลห้องหรือ Booking ของห้องเดียวกันต้องล็อก Room แถวเดียวกันก่อน เพื่อให้ตรวจสถานะ ความจุ การลบ และคำขอพร้อมกันได้สอดคล้องกัน หลังได้ Lock แล้วให้อ่านสถานะ Booking ล่าสุดอีกครั้ง การ Deadlock/Timeout ให้ Rollback แล้วคืน 503 ที่ลองใหม่ได้ ไม่ปล่อย Error ฐานข้อมูลตรง ๆ

Search เป็นข้อมูล ณ เวลาค้นหา ไม่ใช่การถือสิทธิ์จอง จึงอาจค้นพบห้องว่างแต่สร้างการจองแล้วได้ 409 ระบบต้องตรวจซ้ำใน Transaction เสมอ

ข้อต่อที่ทีมต้องใช้เหมือนกัน

เจ้าของ	ส่งมอบให้ทีม
คนที่ 1	Guard ให้ request.user = { id, role }; Shared Error schema/Validation; เพื่อน Mock ผู้ใช้ทดสอบได้
คนที่ 2	Room schema/Seed; วิธีอ่านห้องใน Transaction; การแก้ห้องใช้ Lock เดียวกับ Booking
คนที่ 3	กติกา overlap และ blocking statuses ให้ Search เรียกใช้ร่วมกัน; Booking/Notification schemas
ทุกคน	UUID/Enum/DTO ตรงกัน; Swagger เป็นแหล่ง API contract ในโค้ด; เปลี่ยน Contract ต้องแจ้งพร้อมแก้ Test

กรณีทดสอบขั้นต่ำก่อนรวมงาน

- Auth: สมัคร/Login/Logout แล้ว Token เดิมเรียก Protected API ไม่ได้; User แก้ role ตัวเองไม่ได้
- สิทธิ์: User A ดู/ยกเลิกของ B ได้ 404; User เรียก Admin route ได้ 403; Admin ดูและเปลี่ยนของผู้อื่นผ่าน Admin route ได้
- วันที่: วันที่ไม่มีจริง/คืนศูนย์/วันเข้าในอดีตได้ 400; วันออกเท่ากับวันเข้าของรายการถัดไปไม่ทับซ้อน
- ห้องว่าง: Pending/Approved/Paid กันห้อง; Cancelled ไม่กัน; Search และ Create ใช้เงื่อนไขเดียวกัน
- พร้อมกัน: ส่งจองห้องเดียวกันช่วงเดียวกัน 2 คำขอ ต้องสำเร็จเพียงหนึ่งรายการ อีกคำขอได้ 409
- สถานะ: อนุมัติ/บันทึก Paid วันเข้าได้; ยกเลิกวันเข้าไม่ได้; ส่งสถานะเดิมซ้ำไม่เกิด Notification ซ้ำ

- ข้อมูล: เปลี่ยนราคาห้องไม่เปลี่ยนยอดจองเดิม; Rollback แล้วไม่เหลือ Booking/Notification บางส่วน

ก่อนเริ่ม Sprint 2 ให้ทั้งสามคนยืนยัน Endpoint/DTO, วันที่แบบ [เข้า, ออก), สถานะที่กันห้อง, เงื่อนไขยกเลิก/วันเข้า, PENDING ไม่หมดอายุ, การลบห้อง และ Logout จากนั้นบันทึกการแก้ไขใน Swagger และ README ให้ตรงกัน

9 เริ่มงานร่วมกันใน Sprint 1

ใช้ GitHub repository กลางหนึ่งอัน คนที่ 1 สร้าง Repo และเชิญอีกสองคนเป็น Collaborator จากนั้น Push โครง NestJS พร้อม .gitignore, .env.example และ README ให้ทุกคน Clone ฐานเดียวกัน เก็บ Repo เป็น Private ในช่วงพัฒนาและปรับสิทธิ์ตามคำแนะนำการส่งงานของรายวิชา

ตกลง Node.js รุ่นเดียวกัน ใช้ npm และ Commit package-lock.json เพื่อให้ติดตั้งด้วย npm ci ได้ตรงกัน ห้าม Commit .env, Secret หรือ node_modules ก่อนทุกคนเริ่มแก้ไฟล์ ให้ยืนยันว่าโครงเริ่มต้น Build และรัน Test ได้

แยก Branch ตามงาน เช่น feat/auth, feat/rooms และ feat/bookings ใช้ Pull Request ให้เพื่อนตรวจอย่างน้อยหนึ่งคนก่อนรวม main แล้ว Pull main ล่าสุดก่อนเริ่มงานใหม่ หลีกเลี่ยงการ Push ทับงานของผู้อื่น

ระหว่างคนที่ 1 ตั้ง Repo คนที่ 2 ออกแบบ Room/Database และคนที่ 3 ออกแบบ Booking/Notification พร้อมกติกาการจอง เมื่อฐาน Repo พร้อม ทุกคนเริ่มโมดูลตัวเองได้โดยใช้ Mock/Seed ตาม Contract

ไฟล์กลาง เช่น package.json, app.module.ts, ORM schema และ Migration ต้องแจ้งก่อนแก้ คนที่ 2 ประสานการรวม Schema/Migration แต่ทุกคนรับผิดชอบข้อมูลโมดูลตนเอง เมื่อ API Contract เปลี่ยนให้ระบุใน PR และแจ้งทั้งทีมก่อนรวม